Computers and Electrical Engineering 37 (2011) 973–981

Contents lists available at SciVerse ScienceDirect

Computers and Electrical Engineering

journal homepage: www.elsevier.com/locate/compeleceng

An extended matrix encoding algorithm for steganography of high embedding efficiency ☆

Li Fan [a,*], Tiegang Gao [b], Qunting Yang [a], Yanjun Cao [a]

[a] College of Information Technical Science, Nankai University, 94 Weijin Road, Nankai District, Tianjin 300071, China
[b] College of Software, Nankai University, 94 Weijin Road, Nankai District, Tianjin 300071, China

ARTICLE INFO

Article history:
Received 11 November 2010
Received in revised form 21 August 2011
Accepted 23 August 2011
Available online 16 September 2011

ABSTRACT

F5 steganography is much different from most of LSB replacement or matching steganographic schemes, because matrix encoding is employed to increase embedding efficiency while reducing the number of necessary changes. By using this scheme, the hidden message inserted into carrier media imperceptibly can be transferred via a more secure subliminal channel. The embedding domain is the quantized DCT coefficients of JPEG image, which makes the scheme be immune to visual attack and statistical attack from the steganalyst. Based on this effective scheme, an extended matrix encoding algorithm is proposed to improve the performance further in this paper. By changing the hash function in matrix encoding and converting the coding mode, the embedding efficiency and embedding rate get increased to a large extent. Eventually, the experimental results demonstrate the extended algorithm is superior to the classic F5 steganography.

© 2011 Elsevier Ltd. All rights reserved.

1. Introduction

In 1983, Simmons presented the famous "Prisoners' Problem" to illustrate the model of covert communication [1]. It is pointed out that steganography is the art and science of writing the secret content inside cover media and transferring the stego media from the sender to intended recipient through a subliminal channel without arousing the suspicion of adversary. The presence of hidden information is supposed to be undetectable. If the fact that communication is taking place is revealed, the steganography is cracked no matter whether or not the hidden information is exposed. Thus, compared with other related techniques such as watermarking, the property of covertness plays an important role in the stegosystem.

In order to make stegotext seemingly innocent, the confidential message is usually embedded into the redundant parts of cover media. For digital image, the least significant bit plane in spatial domain is one kind of these parts which look like completely random and noisy. The modification of LSB will not cause noticeable change of the appearance of image. Many LSB-based data hiding techniques are proposed in recent years [2,3]. However, the spatial domain-based methods are vulnerable to pixel alteration and visual attack. In order to enhance the robustness of stegosystem, it is necessary to insert the secret message into more inconspicuous areas such as transform domain. As a result, a great number of methods based on the DCT-domain of JPEG image are developed. Derek Upham's JSteg was probably the first publicly available steganographic system for JPEG images [4]. This method is essentially a carbon copy of the LSB substitution algorithm in spatial domain. The least-significant bit of DCT coefficients is sequentially replaced with the secret message. Since the replacement only happens on two adjacent coefficients, it will cause a statistically obvious POVs (pairs of values) problem which can be successfully detected by χ^2-test proposed by Westfeld and Pfitzmann [5]. To counteract this situation, Outguess0.1 is developed to improve the encoding step

☆ Reviews processed and approved for publication to the Editor-in-Chief Dr. Manu Malek.

* Corresponding author. Tel.: +86 22 23498935; fax: +86 22 23500526.
E-mail addresses: fanlihlm@gmail.com (L. Fan), gaotiegang@gmail.com (T. Gao), quntingyang@gmail.com (Q. Yang), caoyanjun528411@gmail.com (Y. Cao).

0045-7906/$ - see front matter © 2011 Elsevier Ltd. All rights reserved.
doi:10.1016/j.compeleceng.2011.08.006

by scattering the embedding locations over the entire DCT-domain according to a pseudo-random number generator [6]. The χ^2-test for JSteg cannot detect the randomly distributed data successfully. However, it does not last so long. After the release of the algorithm, steganalysts find a fatal flaw that leaves statistical artifacts in the stegogrammes. The creator of Outguess0.1 releases a revised version soon. The new algorithm called Outguess0.2 attempts to ensure that the statistical properties of the cover image can be maintained after encoding [7]. Shortly, Fridrich et al. use the discontinuity of the border between two adjacent 8 by 8 blocks to accurately estimate the length of the hidden data embedded with Outguess0.2 [8]. As an alternative to the Outguess0.2 algorithm, Westfeld designs F3 algorithm which is considered more secure [9]. Unlike the classic LSB embedding, the method compares the absolute value of the coefficients with the secret message. If same, then no modification is made. Otherwise, the absolute value of the corresponding DCT coefficient is reduced by 1. By using this embedding strategy, F3 steganography eliminates the POVs problem which exists in JSteg. But the deficiency of F3 is that more even coefficients are introduced into histogram than odd coefficients which makes the distribution of histogram obviously unnatural. Hence, an improvement called F4 is developed to deal with the unstructured frequency distribution of histogram. In F4, even-negative coefficients and odd-positive coefficients represent 1 and odd-negative coefficients and even-positive coefficients stand for 0. If the value of secret bit is the same as the symbolic coefficient, then no changes happen. Otherwise, the absolute value of the corresponding DCT coefficient is reduced by 1. With the help of this embedding strategy, the histogram of stego image looks statistically similar to a clean image. In addition, there are still many steganographic algorithms proposed such as model-based [10,11], Perturbed Quantization [12], YASS [13] and so on [14–18].

Nonetheless, some of these steganographic algorithms only consider the embedding capacity but neglect the embedding efficiency and security. Some of them just work oppositely. In this paper, an extended algorithm is proposed by improving the matrix encoding of F5 stegosystem to increase the embedding efficiency, whilst the embedding rate is largely improved as well. Some changes in hash function in matrix encoding are performed to hide more secret bits into definite number of cover bits. By exploiting the n-layer extension, we convert the triple $(d_{\max},n,k)$ to quad $(d_{\max},n,k,L)$ to increase the embedding efficiency and embedding rate at the same time. In order to indicate the current number of layer, the symbol positions are deployed in each embedding process. This mechanism makes the receiver realize blind detection effectively. The remainder of this paper is organized as follows. In Section 2, the standard matrix encoding is described. The details of the proposed extended algorithm are presented in the next section. In Section 4, some simulation results and analysis are given, and final conclusion is provided in Section 5.

2. Matrix encoding

Based on the steganographic schemes JSteg, F3 and F4, the scheme F5 is developed by Westfeld [9]. In F5 algorithm, the secret message is inserted by modifying corresponding LSB positions of quantized DCT coefficients of JPEG image as well. However, the main difference is that matrix encoding, in F5, is employed to implement the insertion and detection of secret message instead of classic LSB replacement or matching methods. Additionally, the technique of permutative straddling is used to scatter the secret message over the whole carrier media which enhances the security of stegosystem.

Matrix encoding, as a lower rate method introduced by Crandall [19], improves the embedding efficiency to a great extent in comparison with the classic LSB modification methods. The embedding efficiency, herein, refers to how many bits of secret message can be loaded by one change taken place in carrier media. With regard to LSB method, assuming that the secret message and the values at the positions to be changed subject to the 0–1 uniform distribution, the position to be embedded will be modified with a probability of 0.5 on average. In other words, we have an embedding efficiency of 2 bits per change. If the whole cover image is fully embedded, 50% of carrier data will be changed. That results in a high possibility of statistical steganalysis. As is well known, the most obvious way to reduce the possibility of detection is to reduce the density of changes in the cover image. Matrix encoding not only decreases the necessary number of changes (i.e. the change density), but also increases the embedding efficiency.

The detailed process of encoding is described as follows: first of all, implement JPEG lossy compression on the carrier image and obtain the quantized DCT coefficients for embedding [20]. Afterwards, the LSB plane of quantized DCT coefficients is partitioned into many embedding cells which are in the form of vector $a = a_1a_2\cdots a_n$ with the length of n. The coding implemented on each embedding cell is denoted by an ordered triple $(d_{\max},n,k)$. Where, n is the number of modifiable bit positions in a embedding cell, namely, the length n. k is the bit length of secret message $w = w_1w_2\cdots w_k$ to be embedded into one cell. Also, an embedding cell with n positions will be changed in not more than $d_{\max}$ positions to embed k bits secret message. F5 implements matrix encoding only for $d_{\max} = 1$. A hash function f is defined as Formula (1) to map n bits cover data a into k bits binary string.

$$f : f(a) = \bigoplus_{i=1}^{n} a_i \cdot i, \tag{1}$$

where a_i denotes the i-th position of cell a. i is the corresponding index number of the bit position and is in binary form during the operation. The bit length of binary i is selected as the same size as secret message w. Subsequently, implement XOR operation on the value of hash function and secret message w to obtain a decimal number y.

$$y = w \oplus f(a) \tag{2}$$

Finally, an embedded stego data a' is generated by logically flipping the y-th bit position of cell a. In special case, the carrier cell a will be left intact when $y = 0$.

$$a' = \begin{cases} a & \text{if } y = 0 \\ a_1 a_2 \cdots \bar{a}_y \cdots a_n & \text{otherwise} \end{cases} \tag{3}$$

where, $\bar{a}_y$ is the negation of a_y.

In extraction phase, the receiver would retrieve the secret message w by directly putting the stego cell a' into the same hash function f. The proof is discussed in Appendix A. The detailed procedure of matrix encoding is illustrated in Fig. 1.

For an embedding cell, the change density D refers to the proportion of altered bit position. Neglecting shrinkage, we can calculate the change density depending on Formula (4):

$$D = \frac{\frac{n}{n+1} \cdot 1 + \frac{1}{n+1} \cdot 0}{n} = \frac{1}{n+1} \tag{4}$$

We can also get another important performance measure of steganographic algorithm called the embedding rate R:

$$R = \frac{k}{n} \tag{5}$$

Using the change density and embedding rate, we can define the embedding efficiency E which indicates the average bit number of embedded secret message per change:

$$E = \frac{R}{D} = \frac{n+1}{n} \cdot k \tag{6}$$

Table 1 shows the theoretic values of the change density, the embedding rate and embedding efficiency in according to various of (n,k) pairs. The values are calculated by using Formulas (4)–(6).

In F5 algorithm, the value of n and k must satisfy the equation $n = 2^k - 1$. Because k-bit binary sequence has 2^k potential states in all. It is inevitable that the binary carrier cell has the length of $2^k - 1$ to express all states of k bits secret message by means of modifying only 1 bit or keeping unchanged. That is to say, the length of embedding cell increases with the increase of the length of embedded secret message in manner of exponential growth. Meanwhile, the alteration of carrier cell maintains 1-bit-change. Thus, the embedding rate will become lower and lower with increase of efficiency. The cost of higher efficiency is the lower rate. This regulation not only reveals how matrix encoding works, but also naturally inspires us that we can improve both the embedding efficiency and the embedding rate at the same time if k increases in the context of fixing on n and the number of change.

3. Extended algorithm

How to perform the independent increase of k? In matrix encoding, the hash function maps n bits carrier data into a certain length of binary sequence which depends upon the bit length of index i. And the bit length of i is chosen as the same size as that of secret message w. Given the extension of the length of i is realizable, we can finally embed more bits of secret message into one cell. Taking the cell (1,3,2) as an example, 2-bit secret message will be embedded into 3-bit embedding cell by altering only 1-bit position of the cell. The length of i is two. If the length of i is extended to be 3-bit, we can take one more bit of secret message to implement exclusive-or with the binary result of hash function. However, the problem shows up. The result of XOR operation, namely, the index y which will be used to indicate the position to be changed is out of the range. We may get $y = (101)_2 = 5$ in that case, but it is impossible to find out the fifth bit position for a cell of 3-bit length. Essentially, modifying only 1 bit or keeping unchanged in the carrier cell with 3-bit length can only express four kinds of secret code. These kinds of code are named as '00', '01', '10', '11' with the length of $\log_2 4$. The secret code extended to 3 bits has 2^3 states in all. Thus, there is no way to embed all of eight kinds of code into 3-bit cell by using matrix encoding algorithm.

Actually, there is indeed a way to extend but needed to select some extended codes elaborately. The extension seems to be conditional. Since 3-bit modifiable cell is only able to express four states, we still have a half opportunity to extend by selecting four extended codes to embed from eight codes. During evaluating the result of hash function, we can simply multiply the index i by 2 to extend 1 bit where we call it 1-layer extension. In this case, the codes '00', '01', '10', '11' are extended to '000', '010', '100', '110'. In a similar way, we can multiply i by 2^2 to extend 2 bits called 2-layer extension. The rest may be deduced by analogy. L-Layer extension is performed by multiplying i by 2^L. Due to the closure property of XOR operation, the

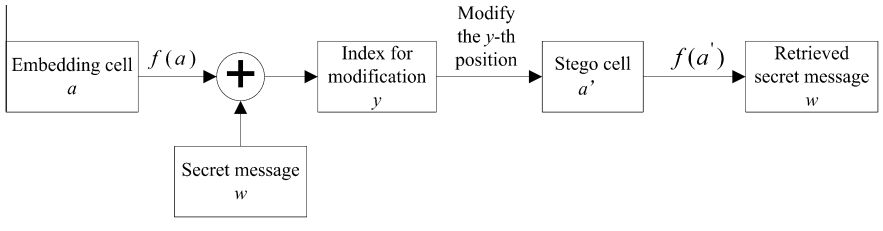


Fig. 1. The block diagram of matrix encoding.

Table 1
The performance of matrix encoding.

(*n*,*k*)	*D* (%)	*R* (%)	*E* (bit)
(1,1)	50	100	2
(3,2)	25	66.67	2.67
(7,3)	12.5	42.86	3.43
(15,4)	6.25	26.67	4.27
(31,5)	3.13	16.13	5.16
(63,6)	1.56	9.52	6.09
(127,7)	0.78	5.51	7.06
(255,8)	0.39	3.14	8.03
(511,9)	0.2	1.76	9.02

secret message with more than 2-bit length can be embedded into 3-bit cell, provided that the secret code is equal to any one of the specific extended codes. The mode of extension is illustrated in Fig. 2.

For extended algorithm, the coding mode implemented on the embedding cell is redefined by a quad $(d_{\max},n,k,L)$, where the new parameter L denotes the maximum of extension layer. Firstly, take out $(k+L)$-bit secret code $w = w_1w_2\cdots w_k\cdots w_{k+L}$ from the whole secret message sequence to test if the secret code matches a specific extended code in the L-th layer. The matching method is to test whether $\mathrm{mod}(w,2^L)=0$ is true. If the remainder equals to zero, the extension layer of current cell l_{crt} is L and a $(k+L)$-bit secret data will be able to be embedded successfully. If not, then continue to test if the prior $(k+L-1)$-bit secret code $w = w_1w_2\cdots w_k\cdots w_{k+L-1}$ matches a specific extended code in the $(L-1)$-th layer by testing the result of $\mathrm{mod}(w,2^{L-1})$. If $\mathrm{mod}(w,2^{L-1})=0$ is true, then the current extension layer is $l_{crt}=L-1$ and the secret code $w = w_1w_2\cdots w_k\cdots w_{k+L-1}$ will be embedded into this cell. But if not, continue to do this kind of test until we find out a matching code in a certain layer or there is no matching code in all extension layers. In latter case, the extended algorithm rolls back to the standard matrix encoding. The final embeddable secret code is in the form of $w = w_1w_2\cdots w_k\cdots w_{k+l_{crt}}$. If no extension takes place, the layer of current cell is $l_{crt}=0$.

In extended algorithm, the hash function is updated as Formula (7):

$$f: f(a) = \bigoplus_{i=1}^{n} a_i \cdot (i \cdot 2^{l_{crt}}) \tag{7}$$

Subsequently, implement XOR operation on the result of hash function and secret message $w = w_1w_2\cdots w_k\cdots w_{k+l_{crt}}$ to obtain a decimal number y. At the moment, the range of index y has already been extended. We must shrink it to n by making y divided by a coefficient $2^{l_{crt}}$.

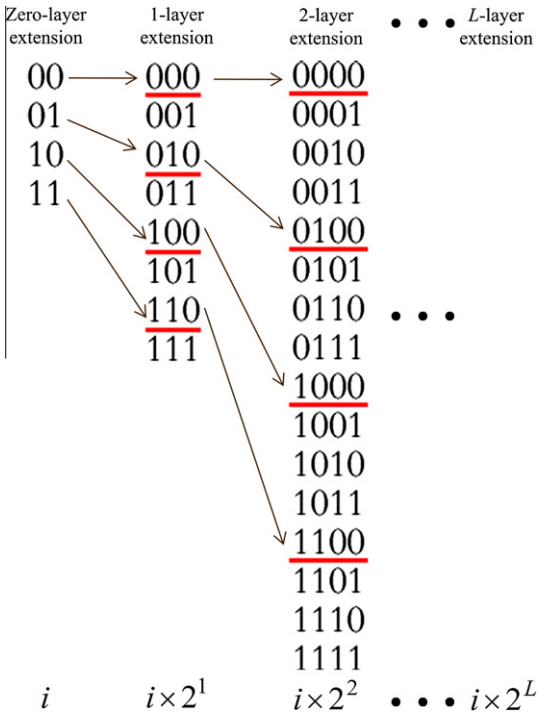


Fig. 2. The chart of the extension mode.

$$y = \frac{w \oplus f(a)}{2^{l_{crt}}}, \tag{8}$$

where the result of $w \oplus f(a)$ is expressed as a decimal number. Eventually, we obtain a stego cell a' by negating the y-th position in carrier cell a.

$$a' = \begin{cases} a & \text{if } y = 0 \\ a_1 a_2 \cdots \bar{a}_y \cdots a_n & \text{otherwise} \end{cases} \tag{9}$$

From the above statement, it is implied that the introduction of extension mechanism raises a new problem to the receiver in detection process. The coding quad ($d_{\max}$,n,k,L) can be confirmed and shared by the sender and the receiver before the start of the communication. Since the current layer l_{crt} is relative to the content of secret message, the receiver cannot predict this parameter definitely. Accordingly, the sender has to transfer l_{crt} to the receiver in the embedding process. We decide to append a symbol $s = s_1 s_2 \cdots s_m$ to the stego cell $a' = a_1 a_2 \cdots \bar{a}_y \cdots a_n$ to mark the layer l_{crt}. Because the value of l_{crt} is fallen into the closed interval of $[0,L]$, the length of symbol m can be calculated by Formula (10). We use the binary number of l_{crt} to assign the symbol s.

$$m = \lceil \log_2(L+1) \rceil \tag{10}$$

Thus, the new stego cell c with the length of $(n+m)$ is composed of two parts, namely, data part and symbol part (i.e. the cell is reformed as $c = a's = a_1 a_2 \cdots \bar{a}_y \cdots a_n s_1 s_2 \cdots s_m$). In extraction phase, the receiver firstly take out the symbol part of the stego cell c and calculate the layer l_{crt}.

$$l_{crt} = dec(s_1 s_2 \cdots s_m) \tag{11}$$

Eventually, the extended secret data $w = w_1 w_2 \cdots w_k \cdots w_{k+l_{crt}}$ is retrieved by putting the data part of the stego cell c into the updated hash function $f : f(a) = \bigoplus_{i=1}^{n} a_i \cdot (i \cdot 2^{l_{crt}})$.

$$w = f(a') \tag{12}$$

The detailed procedure of the extended matrix encoding is shown in Fig. 3.

To be more clear, we take the coding mode of (1,7,3,2) as an example to show how the extended algorithm works. Assume that the carrier data is $a = 1101010$, the secret data taken from the whole secret sequence is $w = 11001$.

First of all, the sender tests if $\mathrm{mod}(dec(11001), 2^2) = 0$ is true. Due to $\mathrm{mod}(dec(11001), 2^2) = 1$, the sender continue to test the shorter secret data $w = 1100$. Since $\mathrm{mod}(dec(1100), 2^1) = 0$ is true, it is confirmed that the secret data $w = 1100$ can be embedded into the carrier data and the current layer $l_{crt} = 1$.

Secondly, calculate the length of symbol $m = \lceil \log_2 3 \rceil = 2$ and assigns the symbol $s = 01$.

Thirdly, calculate the hash function with the carrier data $a = 1101010$ as shown as follows:

$$\begin{aligned} & 0010 \\ & 0100 \\ & 1000 \leftarrow a_i \cdot (i \cdot 2^1) \\ f(a) : & \oplus 1100 \\ & \overline{0010} \end{aligned} \tag{13}$$

Finally, calculate index $y = (w \oplus f(a))/2 = ((1100)_2 \oplus (0010)_2)/2 = 7$ and flip the seventh bit position of carrier data $a = 1101010$ to generate a stego data $a' = 1101011$. Up to now, a stego cell $c = 110101101$ is obtained.

In detection process, the receiver firstly takes the symbol $s = 01$ from the stego cell and calculates the current layer $l_{crt} = dec(01) = 1$. Subsequently, calculate hash function with the stego data $a' = 1101011$ to retrieve the secret data $w = 1100$ as follows:

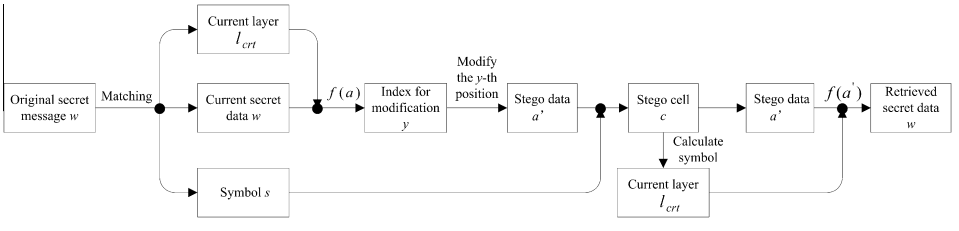


Fig. 3. The flowchart of the extended matrix encoding.

$$\begin{array}{r} 0010 \\ 0100 \\ 1000 \\ 1100 \\ f(a'): \oplus 1110 \\ \hline 1100 \end{array} \tag{14}$$

4. Simulation and analysis

In order to test the performance of the proposed algorithm, we employ a meaningful binary logo image with 64 by 64 pixels as the secret message and hide it into the carrier image Lena with 256 by 256 pixels shown in Fig. 4(a and b). The corresponding stego image is shown in Fig. 4(c).

The PSNR of original carrier image and stego image is 70.33. It is indicated that the extended method has a strong covertness. Fig. 5 illustrates the comparison between the embedding efficiency of standard matrix encoding and that of extended algorithm with three different maximal extension layer (respectively $L = 7,15,31$).

It can be seen from the figure that the embedding efficiency of the extended algorithm is higher than that of matrix encoding when the ratio of k to n is larger. However, when the ratio is less than 3/7, the efficiency of extended algorithm with $L = 7$ starts to be lower than that of standard algorithm. When the ratio is less than 4/15 and 5/31, the efficiency of extended algorithm with $L = 15$ and 31 starts to be lower than that of standard algorithm, respectively. Although the extended algorithm increases the bit number of embedded secret message, the modification for setting symbol positions affect the embedding efficiency because these changes do not load any secret message. Actually, the bit number of average changes for setting m-bit symbol is $m/2$ bits. The proof of this conclusion is presented in Appendix B.

For every embedding cell, there are average $m/2$ bits modification used to set symbol rather than loading secret message. With the decrease of the ratio of k to n, the length of carrier data n becomes large which leads to the significant reduction of the number of embedding cells generated by partitioning the LSB plane of quantized DCT coefficients. In other words, the proportion of altered bits in carrier data is very small. In this case, the influence of symbol modification emerges and the cost of changing without loading effective information cannot be neglected. Oppositely, when the ratio of k to n is large, more secret data can be embedded and a large number of changes exist inside the cover data. The influence of symbol modification is relatively little. Although we can improve the embedding efficiency by increasing the maximal extension layer L, the increase of L will result in the increase of length of symbol which has a negative impact on the efficiency. Thus, we should find out the optimal tradeoff to confirm the coding mode $(1,n,k,L)$ according to the size and type of secret message. Taking the coding mode $(1,3,2,L)$ for example, a new logo image illustrated in Fig. 6(a) will be embedded into the test image Lena using the extended scheme. It can be seen from Fig. 6(b) that the values of embedding efficiency vary in according to the maximum of extension layer L when n and k are specific. In this case, the maximum of embedding efficiency is obtained when $L = 15$. When L is less than 15, the secret bits are not extended adequately. Contrarily, a larger L leads to the increase of the length of symbol which has a negative impact on the embedding efficiency. Obviously, the coding mode $(1,3,2,15)$ is the best choice in this instance.

In matrix encoding, the length of secret message k is always smaller than the length of carrier cell n. It means the embedding rate is smaller than 100% for ever. However, it is possible for the extended algorithm that the length of secret message $(k + l_{crt})$ is larger than the length of carrier cell $(n + m)$ due to the extension. Theoretically, the embedding rate can exceed 100% and the maximum value can be close to $R_{\max}$.

$$R_{\max} = \frac{L_s}{n + \lceil \log_2(L_s - k + 1) \rceil}, \tag{15}$$



(a) secret message (b) carrier image (c) stego image

Fig. 4. The performance of the extended algorithm.

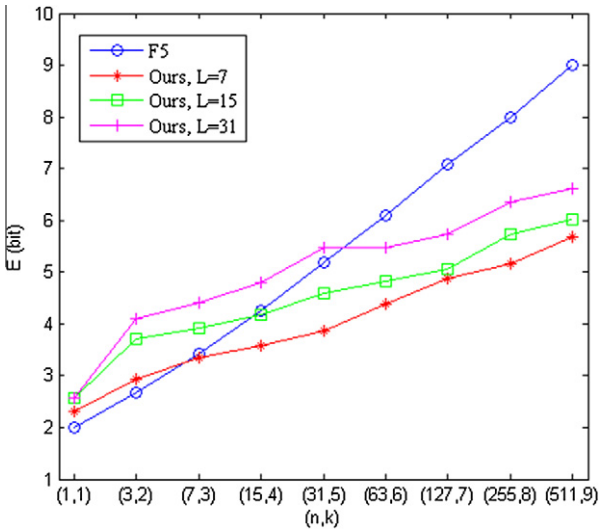
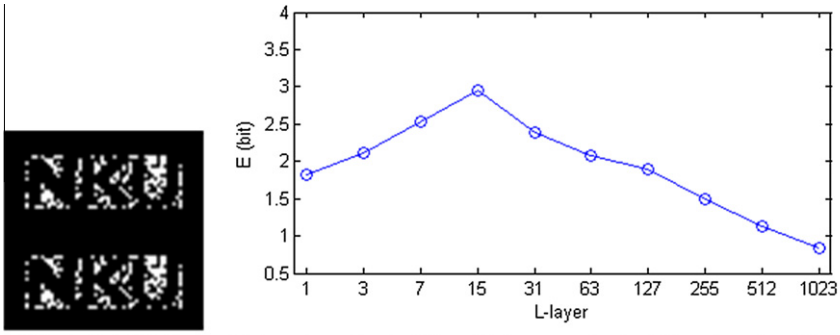


Fig. 5. The comparison of the embedding efficiency.



(a) new secret logo (b) the values of embedding efficiency varying in different number of layer

Fig. 6. The relationship between L and E when n and k are specific.

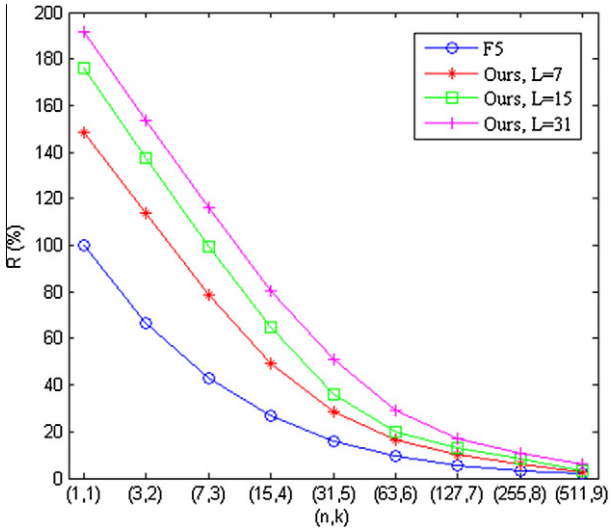


Fig. 7. The comparison of the embedding rate.

where L_s denotes the length of the whole secret message. Fig. 7 shows the comparison between the embedding rate of standard matrix encoding and that of extended algorithm with three different maximal extension layer (respectively L = 7,15,31).

5. Conclusion

Many steganographic algorithms offer a high capacity for hidden messages but neglect the embedding efficiency. In fact, fewer modifications could guarantee the high covertness of the stegosystem. F5 steganography employs matrix encoding to obtain higher efficiency than classic LSB replacement or matching methods. However, with the increase of the efficiency, the embedding rate is reduced rapidly. In high efficiency case, a small amount of secret message can be embedded which leads to a waste of carrier data. The extended matrix encoding algorithm proposed in this paper uses n-layer extension to improve the embedding efficiency and rate simultaneously. In the application of high ratio of k to n, the extended algorithm can obtain a higher efficiency. Moreover, the extended algorithm makes the embedding rate exceed 100% theoretically and realizes a high capacity whilst performing a high efficiency.

However, from some experimental results, e.g. in Fig. 5, it can be seen that the embedding efficiency of improved scheme is not always higher than F5 all the time. When the ratio of k to n becomes small, the embedding efficiency gets decreased due to the introduction of symbol bits. The reason of this phenomenon is mainly that the changes for setting symbols do not load any secret message and a random-type secret message cannot always be extended as well (actually with the possibility of 0.5). For the new method, this problem seems inevitable. In spite of this, our scheme is still important and meaningful because we also use binary images which are full of consecutive black pixels '00000000...' like the logos in Figs. 4 and 6. This kind of secret message has more opportunities to be extended with a large number of extension layers. The experimental results demonstrate that the extended algorithm has a high covertness, a high embedding efficiency and a high embedding rate in the most cases. And the scheme is effective enough to be applied to the area of covert communication.

Acknowledgments

The authors would like to thank the support from National Science Foundation of China (#60873117) and Key Program of Natural Science Fund of Tianjin (Grant #11JCZDJC16000).

Appendix A

In this part, we prove that the receiver would figure out the secret message w by directly putting the stego cell a' into the same hash function f. The details of proof are depicted as follows:

$$f(a') = \mathop{\oplus}_{i\neq y} a_i i \oplus \bar{a}_y y = \mathop{\oplus}_{i\neq y} a_i i \oplus (1 \oplus a_y)(w \oplus f(a)) = \mathop{\oplus}_{i\neq y} a_i i \oplus w \oplus f(a) \oplus a_y w \oplus a_y f(a)$$
$$= \mathop{\oplus}_{i\neq y} a_i i \oplus w \oplus f(a) \oplus a_y y = f(a) \oplus w \oplus f(a) = w$$

Appendix B

In this section, it will be proven that the bit number of average changes for setting m-bits symbol is $m/2$ bits.

Given that the carrier data subject to uniformly distributed, the statement of figuring out the bit number of average changes for setting binary symbol $s_1 s_2 \cdots s_m$ is equivalent to finding the average Hamming distance between the fixed code $s_1 s_2 \cdots s_m$ and all of other codes in the m length code space. There are 2^m codes in all in the code space. The number of the codes whose Hamming distance is 1 bit from the $s_1 s_2 \cdots s_m$ is C_m^1. And the number of the codes whose Hamming distance is 2-bit from the $s_1 s_2 \cdots s_m$ is C_m^2. Similarly, the number of the codes whose Hamming distance is m-bit from the $s_1 s_2 \cdots s_m$ is C_m^m. Thus, the average Hamming distance is $\frac{1}{2^m}\sum_{i=1}^{m} i \cdot C_m^i$. As a result of the binomial theorem:

$$(a+b)^m = C_m^0 a^m + C_m^1 a^{m-1} b + C_m^2 a^{m-2} b^2 + \cdots + C_m^{m-1} a b^{m-1} + C_m^m b^m \tag{16}$$

Let a = 1, then

$$(1+b)^m = C_m^0 + C_m^1 b + C_m^2 b^2 + \cdots + C_m^{m-1} b^{m-1} + C_m^m b^m \tag{17}$$

Calculate the derivative of b on both sides of Formula (16), then

$$m(1+b)^{m-1} = C_m^1 + 2C_m^2 b + \cdots + (m-1)C_m^{m-1} b^{m-2} + mC_m^m b^{m-1} \tag{18}$$

Let b = 1, then

$$m \cdot 2^{m-1} = C_m^1 + 2C_m^2 + \cdots + (m-1)C_m^{m-1} + mC_m^m = \sum_{i=1}^{m} i \cdot C_m^i \tag{19}$$

Thus, the bit number of average changes for setting m-bits symbol is

$$\frac{1}{2^m}\sum_{i=1}^{m} i \cdot C_m^i = \frac{1}{2^m} \cdot m \cdot 2^{m-1} = \frac{m}{2} \tag{20}$$

References

[1] Simmons GJ. The prisoners' problem and the subliminal channel. In: Advances in cryptology: proceedings of crypto 83, New York; 1984. p. 51–67.
[2] van Schyndel RG, Tirkel A, Osborne CF. A digital watermark. In: Proc. of int. conf. on image processing, Austin; November 1994. p. 86–9.
[3] Franz E, Jerichow A, Moller S, Pfitzmann A, Stierand I. Computer based steganography: how it works and why therefore any restrictions on cryptography are nonsense, at best. In: Proc. of the 1st international workshop on information hiding, Cambridge; May 1996. p. 7–21.
[4] Zhang Tao, Ping Xijian. A fast and effective steganalytic technique against JSteg-like algorithms. In: Proc. 8th ACM symp. on applied computing, Florida; March 2003. p. 307–11.
[5] Westfeld A, Pfitzmann A. Attacks on steganographic systems. Lect Notes Comput Sci 2000;1768:61–75.
[6] Provos N. Defending against statistical steganalysis. In: Proc. of the 10th USENIX security symposium, Washington, DC; August 2001. p. 323–35.
[7] Provos N, Honeyman P. Hide and seek: an introduction to steganography. IEEE Secur Privacy 2003;1:32–44.
[8] Fridrich J, Goljan M, Hogea D. Attacking the OutGuess. In: Proc. of the 3rd information hiding workshop on multimedia and security, Juan-les-Pins; December 2002. p. 3–6.
[9] Westfeld A. F5-a steganographic algorithm: high capacity despite better steganalysis. Lect Notes Comput Sci 2001;2137:289–302.
[10] Sallee P. Model-based steganography. In: Proc. of the 2nd international workshop on digital watermarking, Seoul; October 2003. p. 154–67.
[11] Sallee P. Model-based methods for steganography and steganalysis. Int J Image Graphics 2005;5(1):167–90.
[12] Fridrich J, Goljan M, Soukal D. Perturbed quantization steganography with wet paper codes. In: Proc. of ACM multimedia workshop, Magdeburg; September 2004. p. 4–15.
[13] Solanki K, Sarkar A, Manjunath B. YASS: yet another steganographic scheme that resists blind steganalysis. Lect Notes Comput Sci 2008;4567:16–31.
[14] Wang Xiangyang, Yang Yiping, Yang Hongying. Invariant image watermarking using multi-scale Harris detector and wavelet moments. Comput Electr Eng 2010;36(1):31–44.
[15] Lu Wei, Lu Hongtao, Chung Fulai. Feature based robust watermarking using image normalization. Comput Electr Eng 2010;36(1):2–18.
[16] Lu Wei, Sun Wei, Lu Hongtao. Robust watermarking based on DWT and nonnegative matrix factorization. Comput Electr Eng 2009;35(1):183–8.
[17] Al-Otum HA, Al-Taba'a AO. Adaptive color image watermarking based on a modified improved pixel-wise masking technique. Comput Electr Eng 2009;35(5):673–95.
[18] Fan Li, Gao Tiegang, Yang Qunting. A novel watermarking scheme for copyright protection based on adaptive joint image feature and visual secret sharing. Inf Control. 2011;7(7):3679–94.
[19] Crandall R. Some notes on steganography. Posted on steganography mailing list; 1998. http://os.inf.tu-dresden.de/~westfeld/crandall.pdf.
[20] Wallace GK. The JPEG still picture compression standard. IEEE Trans Consum Electron 1992;38(1):xviii–xxiv.

Li Fan was born in Tianjin, P.R.C., in 1982. He received the B.S. degree from the College of Information and Communication Engineering, Harbin Engineering University, Harbin, China, in 2005 and the M.S. degree from the College of Software, Nankai University, Tianjin, China, in 2010. He is currently pursuing the Ph.D. degree in the College of Information Technical Science, Nankai University, Tianjin, China. His research interests include digital watermarking, steganography and steganalysis.

Tiegang Gao was born in 1966. He received the B.E. and M.E. degrees in mathematics and applied mathematics from Hebei Teacher's University and Huazhong University of Science and Technology, China, in 1988 and 1990, respectively, and the Ph.D. degree in control theory and control engineering from Nankai University, China, in 2005. Now, he is a Professor at College of Software, Nankai University, China. His main interests of research are multimedia information security, image processing and nonlinear system, now he has authored/coauthored more than 60 papers in these areas.

Qunting Yang was born in Nanyang, Henan, P.R.C., in 1983. He received the B.S. degree from the college of Computer Science, Henan University, Kaifeng, China, in 2006 and the M.S. degree from the College of Software, Nankai University, Tianjin, China, in 2009. He is currently a Ph.D. student in the College of Information Technical Science, Nankai University, Tianjin, China. His research interests include information hiding and encryption algorithm.

Yanjun Cao was born in Shijiazhuang, Hebei, P.R.C., in 1981. He received the B.S. degree from the college of Mathematics and Information, Langfang Teachers College, Langfang, China, in 2006 and the M.S. degree from the College of Computer Science and Software, Hebei University of Technology, Tianjin, China, in 2009. He is currently a Ph.D. candidate in the College of Information Technical Science, Nankai University, Tianjin, China. His research interests include image forensics and information hiding.